

# 華中科技大學

## C 語言程序設計

題目：

基于 SAT 的百分号数独游戏求解程序

专业班级 智科 2402

学 号 U202490035

姓 名 阮云轩

指导教师 许贵平

报告日期 2025 年 10 月 01 日

## 目 录

|          |                            |           |
|----------|----------------------------|-----------|
| <b>1</b> | <b>引言 .....</b>            | <b>1</b>  |
| <b>2</b> | <b>程序设计综合课程设计课程概述.....</b> | <b>2</b>  |
| 2.1      | 课程背景 .....                 | 2         |
| 2.2      | 课程目标 .....                 | 2         |
| 2.3      | 课程任务 .....                 | 2         |
| <b>3</b> | <b>程序总体设计 .....</b>        | <b>3</b>  |
| 3.1      | SAT 求解器.....               | 3         |
| 3.2      | 百分号数独游戏 .....              | 3         |
| 3.3      | 系统结构分析 .....               | 4         |
| <b>4</b> | <b>数据结构和算法详细设计.....</b>    | <b>8</b>  |
| 4.1      | definition.hpp.....        | 8         |
| 4.2      | CNF.cpp .....              | 9         |
| 4.3      | DPLLsolver.cpp.....        | 12        |
| 4.4      | display.cpp .....          | 14        |
| 4.5      | main.cpp .....             | 15        |
| <b>5</b> | <b>程序实现 .....</b>          | <b>16</b> |
| <b>6</b> | <b>复杂度分析 .....</b>         | <b>23</b> |
| 6.1      | 时间复杂度分析 .....              | 23        |
| 6.2      | 空间复杂度分析 .....              | 23        |
| <b>7</b> | <b>小结 .....</b>            | <b>24</b> |
| <b>8</b> | <b>主要参考文献.....</b>         | <b>25</b> |
| 8.1      | 附录一源代码 .....               | 25        |
| 8.2      | 附录二指导参考书目录 .....           | 63        |

## 1 引言

在着手这个“基于 SAT 的百分号数独求解程序”之前，我从未想过，一个看似简单的逻辑游戏背后，竟隐藏着如此复杂的计算理论。这次课程设计对我而言，就像一次从理论到实践的冒险之旅。

在这个过程中，理解和实现 DPLL 算法对我来说是一份十分大的挑战，要把 DPLL 思想变成一个能在计算机中高效运行的代码。反复推敲着每一个约束条件的逻辑表达。

在过程中我深刻地体会到，调试代码的过程，其实也是在调试自己的思维。它强迫我以更严谨、更周密的方式去思考问题。这次经历不仅让我对 NP 问题和搜索算法有了直观的认识，更重要的是，它让我明白，一个完整的程序项目，其核心不仅是实现功能，更是在不断的“发现问题-解决问题”的循环中，构建起一个稳定可靠的系统。

## 2 程序设计综合课程设计课程概述

### 2.1 课程背景

“程序设计综合课程设计”作为计算机科学与技术学院开设的一门综合性实践教学环节。本课程旨在引导学生整合并应用前期所学的编程语言、数据结构与算法知识，通过完成一个规模适中、功能完整的软件项目，实现从零散的语法知识学习到系统性工程实践的过渡。

本设计以“基于 SAT 的百分号数独游戏求解程序”为具体课题，其核心在于实现经典的 DPLL 算法以构建一个完备的 SAT 求解器。该过程要求学生不仅需要深入理解 NP 完全问题与回溯搜索算法的理论基础，更需将其转化为稳定、高效的代码实现。这一实践对于巩固计算理论基础、提升系统编程能力具有重要作用。

### 2.2 课程目标

**深化算法理解与实现能力：**通过亲手实现 DPLL 算法这一经典回溯搜索框架，并对其进行必要的优化，使学生深刻理解算法背后的理论与实现细节

**提升系统设计与编程调试能力：**课程要求设计并实现 SAT 求解器与数独游戏应用之间的完整逻辑链。这要求我们进行合理的模块划分，设计高效的数据结构，并在持续的编码、测试与调试过程中，系统性地提升解决实际编程问题与排除错误的能力。

### 2.3 课程任务

SAT 问题即命题逻辑公式的可满足性问题 (satisfiability problem)，是计算机科学与人工智能基本问题，是一个典型的 NP 完全问题，可广泛应用于许多实际问题如硬件设计、安全协议验证等，具有重要理论意义与应用价值。本设计要求基于 DPLL 算法实现一个完备 SAT 求解器，对输入的 CNF 范式算例文件，解析并建立其内部表示；精心设计问题中变元、文字、子句、公式等有效的物理存储结构以及一定的分支变元处理策略，使求解器具有优化的执行性能；对一定规模的算例能有效求解，输出与文件保存求解结果，统计求解时间。

## 3 程序总体设计

这里我们会针对本次的实验目标列出我们的核心模块和分文件处理，方便我们一步步去代码化和维护本次的项目，也能更方便我们管理和使用

### 3.1 SAT 求解器

(1) 输入功能：用户可以输入参数决定要使用的功能（读取文件，输出 CNF 内容，DPLL 求解，返回等）。用户可以输入 `cnf` 文件的路径，程序将从中读取内容到 CNF 的数据结构中。程序应具备检错功能以防止用户的错误输入，例如在没有读取文件的情况下就进行求解。

(2) 输出功能：程序可以将对 `cnf` 算例文件求解的结果保存到同名的文件中并将运行结果及相关数据顺利输出。

(3) 求解功能：通过调用 DPLL 函数，对读入的 `cnf` 算例文件进行求解，用户理应在这一步时选择变元选取策略。

(4) 界面功能：系统应提供简单的 UI 界面功能用于交互，清屏提供良好的基本使用。

### 3.2 百分号数独游戏

百分号数独游戏部分应具备以下功能：生成数独：每次随机生成一个数独，但难度策略没写。交互功能：可以输入单元格坐标在指定位置填入数字，撤销操作，重新开始游戏，查看答案，或调用 SAT 求解器对数独进行求解。检查功能：程序能在用户游玩时的输入进行检查，如输入的坐标和数据是否在有效范围内，输入是否合法，输入的坐标是否已被初始数独棋盘已经提供的。界面功能：通过合理布局和各种符号的使用，提供一个视觉效果良好的数独面板，并具备清屏功能，返回功能等。DPLL 求解功能：程序应能将生成的数独游戏转化为 SAT 问题，即将局面转化为一个 `cnf` 文件，并调用之前的 SAT 求解器进行求解，将最终各单元格的赋值结果打印到屏幕上。

## 3.3 系统结构分析

本系统致力于实现一个集成了 SAT 求解器与百分号数独游戏的综合应用。系统通过一个简洁的菜单界面与用户交互，核心功能分为两大部分：一是作为基础引擎的 SAT 求解器，能够处理通用的 CNF 公式文件；二是基于该求解器开发的百分号数独游戏模块，实现了从游戏生成、自动求解到用户交互游玩的完整流程。

系统采用模块化设计，除主控模块外，各核心功能均封装在独立的模块中，通过头文件（.h）和实现文件（.cpp）进行组织，确保了代码的清晰性和可维护性。系统主要包含以下功能模块：

### 3.3.1 主控与交互模块 (Display)

作为程序的入口和调度中心，该模块在 `main.cpp` 中实现。它通过循环、分支语句（如 `while`, `switch`）结合系统命令（如 `system("pause")`, `system("cls")`）构建了清晰的文本菜单。该模块负责接收用户的输入指令，并据此调用其他功能模块，是整个系统与用户互动的桥梁。

### 3.3.2 CNF 解析模块 (CNFParser)

该模块由 `CNF` 类和 `CNFParser` 类构成，核心职责是读取并解析标准的 CNF 格式文件。其成员函数 `parser` 接受文件名作为参数，将文件中的子句、变元等信息准确加载到 `CNF` 对象的数据结构中。模块具备基本的错误处理能力，在文件不存在或格式不符时能返回明确的错误信息，为后续求解提供可靠的数据基础。

### 3.3.3 核心求解模块 (DPLL-Solver)

本模块是 SAT 求解器的算法核心，完整实现了经典的 DPLL 回溯搜索算法。其内部主要由三个关键函数协同工作：DPLL 主框架负责控制整体的搜索与回溯流程；变元决策函数依据特定策略（如最大出现频率）选择分支变元；单位传播函数则负责执行单子句规则，对公式进行化简。此外，模块还提供了屏幕输出与文件输出功能，可将求解结果（满足性、变元赋值、求解时间）清晰地展示给用户或保存至 `.res` 文件。

## 3.3.4 百分号数独游戏模块 (XSudoku Game)

本模块是 SAT 求解器的具体应用，将理论算法应用于有趣的百分号数独游戏。它进一步细分为四个子模块：

1. 游戏生成子模块：采用递归回溯算法，首先生成一个合法的数独终盘，特别确保了其满足百分号数独特有的“窗口”与“对角线”约束，然后通过“挖洞”策略生成初始游戏格局。
2. 问题归约子模块：此模块是应用的关键，它将一个具体的百分号数独棋盘状态，按照预定义的编码规则，自动转化为等价的 CNF 公式文件。该过程不仅包含了标准数独的行、列、宫约束，还精确地生成了用于描述额外对角线约束和窗口约束的子句。
3. 自动求解子模块：该子模块直接调用核心的 DPLL 求解器，对归约后生成的 CNF 文件进行求解。求解完成后，并非输出原始的变元赋值，而是设计了一个专用的输出函数，将 SAT 解翻译回用户可读的九宫格数字填充方案。
4. 交互检错子模块 (Check Game)：在用户手动游玩过程中，该模块提供实时验证功能。每当用户填入一个数字，它会立即检查该操作是否违反了数独的基本规则或百分号数独的特殊规则，并在检测到冲突时及时提示用户，引导其进行修正，极大地提升了游戏的可玩性和友好度。

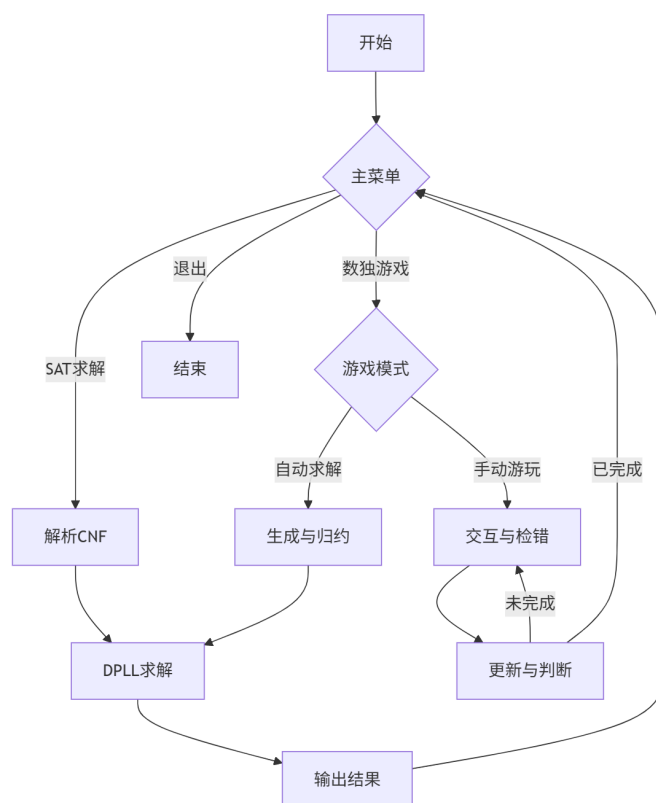


图 3-1 程序流程图

## 流程说明

程序通过主菜单分为 SAT 求解和游戏两大路径



SAT 路径: CNF 解析 → DPLL 求解 → 结果输出

游戏路径:

自动求解: 生成数独 → 转为 CNF → 调用 DPLL 求解

手动游玩: 用户输入 → 实时检错 → 循环直到完成

各功能完成后均返回主菜单

## 4 数据结构和算法详细设计

首先我们会对项目分文件处理，到最后希望的结果是 `main.cpp` 被高效地包装好了只剩简洁的函数。主要的核心头文件有以下，我会在后面一一介绍

### 4.1 definition.hpp

这个 `definition.hpp`(1) 头文件的任务如命名一样十分明了，负责为我们将会用到的函数声名工作，作为核心头文件，包含所有数据结构定义和函数声明，定义了 CNF 公式的数据结构：文字节点、子句节点、CNF 节点，声明了所有模块的函数原型，包含必要的系统头文件和宏定义

Listing 1 definition.hpp

```
1      #include <bits/stdc++.h>
2      #include <windows.h>
3      #include <winnt.h>
4      using namespace std;
5      #pragma once
6
7      #define status int
8      #define OK 1
9      #define TRUE 1
10     #define FALSE 0
11     #define SIZE 9
12
13     //文字节点
14     typedef struct literalNode
15     {
16         int literal;
17         struct literalNode *next;
18     } literalNode, *literalList;
19
20     //子句节点
21     typedef struct clauseNode
22     {
23         literalList head;
24         struct clauseNode *next;
25     } clauseNode, *clauseList;
26
```

```
27      //CNF文件
28      typedef struct cnfNode
29      {
30          clauseList root;
31          int boolCount;
32          int clauseCount;
33      } cnfNode, *CNF;
34
35      //函数定义
36      CNF readCNF(const char* filename);
37      void destroyCNF(CNF cnf);
38      void printCNF(CNF cnf);
39      int is_single_word(literalList L);
40      int find_single(clauseList L);
41      int DestroyClause(clauseList &cL);
42      void Simplify(clauseList &cL, int literal);
43      clauseList CopyCnf(clauseList cL);
44      int ChooseLiteral_1(CNF cnf);
45      int ChooseLiteral_2(CNF cnf);
46      int ChooseLiteral_3(CNF cnf);
47      int Satisfy(clauseList cL);
48      int EmptyClause(clauseList cL);
49      int DPLL(CNF cnf, bool value[], int flag);
50      int SaveResult(int result, double time, double time_, bool value[], char
          fileName[], int boolCount);
```

## 4.2 CNF.cpp

CNF 文件 (2) 解析模块，负责读取和解析.cnf 格式的 SAT 算例文件，将文件内容转换为内存中的链表结构表示，同时也提供 CNF 结构的创建、销毁和打印功能

Listing 2 CNF.cpp

```
1      CNF readCNF(const char* filename) {
2          FILE* file = fopen(filename, "r");
3          if (!file) {
4              printf("无法打开文件: %s\n", filename);
5              return NULL;
6          }
7      }
```

# 华中科技大学课程实验报告

---

```
8         CNF cnf = (CNF)malloc(sizeof(cnfNode));
9         if (!cnf) {
10             fclose(file);
11             return NULL;
12         }
13
14         cnf->root = NULL;
15         cnf->boolCount = 0;
16         cnf->clauseCount = 0;
17
18         char line[1024];
19         clauseList lastClause = NULL;
20
21         while (fgets(line, sizeof(line), file)) {
22             // 跳过注释行和空行
23             if (line[0] == 'c' || line[0] == '\n') {
24                 continue;
25             }
26
27             // 解析问题
28             if (line[0] == 'p') {
29                 char format[10];
30                 sscanf(line, "p %s %d %d", format, &cnf->boolCount,
31                     &cnf->clauseCount);
32                 continue;
33             }
34
35             // 解析子句
36             clauseNode* newClause =
37                 (clauseNode*)malloc(sizeof(clauseNode));
38             if (!newClause) {
39                 fclose(file);
40                 destroyCNF(cnf);
41                 return NULL;
42             }
43
44             newClause->head = NULL;
45             newClause->next = NULL;
46
47             // 如果是第一个子句，设置为根节点
48             if (!cnf->root) {
```

# 华中科技大学课程实验报告

---

```
47         cnf->root = newClause;
48         lastClause = newClause;
49     } else {
50         lastClause->next = newClause;
51         lastClause = newClause;
52     }
53
54     // 解析文字
55     char* token = strtok(line, " \t\n");
56     literalList lastLiteral = NULL;
57
58     while (token) {
59         int literal = atoi(token);
60
61         // 遇到0表示子句结束
62         if (literal == 0) {
63             break;
64         }
65
66         literalNode* newLiteral =
67             (literalNode*)malloc(sizeof(literalNode));
68         if (!newLiteral) {
69             fclose(file);
70             destroyCNF(cnf);
71             return NULL;
72         }
73
74         newLiteral->literal = literal;
75         newLiteral->next = NULL;
76
77         if (!newClause->head) {
78             newClause->head = newLiteral;
79         } else {
80             lastLiteral->next = newLiteral;
81         }
82
83         lastLiteral = newLiteral;
84         token = strtok(NULL, " \t\n");
85     }
86     printf("读取完成! \n");
```

```
87         fclose(file);
88         return cnf;
89     }
```

## 4.3 DPLLsolver.cpp

SAT 求解核心算法模块 (13) 实现了完整的 DPLL 算法框架  
包含三种变元选择策略：

策略 1：选择第一个出现的变元

策略 2：选择出现频率最高的变元

策略 3：选择最短子句中出现的变元

处理单子句传播、公式化简、回溯搜索

Listing 3 DPLLsolver.cpp

```
1      //未优化: 直接选择
2      int ChooseLiteral_1(CNF cnf)
3      {
4          return cnf->root->head->literal;
5      }
6
7
8      //优化2: 选择出现次数最多的文字
9      int ChooseLiteral_2(CNF cnf)
10     {
11         clauseList lp = cnf->root;
12         literalList dp;
13         int *count, MaxWord, max; //
14         // count记录每个文字出现次数,MaxWord记录出现最多次数的文字
15         count = (int *)malloc(sizeof(int) * (cnf->boolCount * 2 + 1));
16         for (int i = 0; i <= cnf->boolCount * 2; i++)
17             count[i] = 0; // 初始化
18         // 计算子句中各文字出现次数
19         for (lp = cnf->root; lp != NULL; lp = lp->next)
20         {
21             for (dp = lp->head; dp != NULL; dp = dp->next)
22             {
23                 if (dp->literal > 0) // 正文字
24                     count[dp->literal]++;
```

```
24         else
25             count[cnf->boolCount - dp->literal]++; // 负文字
26     }
27 }
28 max = 0;
29 // 找到出现次数最多的正文字
30 for (int i = 1; i <= cnf->boolCount; i++)
31 {
32     if (max < count[i])
33     {
34         max = count[i];
35         MaxWord = i;
36     }
37 }
38 if (max == 0)
39 {
40     // 若没有出现正文字,找到出现次数最多的负文字
41     for (int i = cnf->boolCount + 1; i <= cnf->boolCount * 2;
42          i++)
43     {
44         if (max < count[i])
45         {
46             max = count[i];
47             MaxWord = cnf->boolCount - i;
48         }
49     }
50     free(count);
51     return MaxWord;
52 }
53
54 //优化3: 选择最短子句中出现次数最多的文字
55 int ChooseLiteral_3(CNF cnf)
56 {
57     clauseList p = cnf->root;
58     int *count = (int *)calloc(cnf->boolCount * 2 + 1, sizeof(int));
59     int minSize = INT_MAX; // 初始化为大于可能的最大子句长度
60     int literal = 0;
61     clauseList temp = NULL;
62     // 遍历子句,找到最小子句并统计其文字
63     while (p != NULL)
```

```
64         {
65             literalList q = p->head;
66             int clauseSize = 0;
67             while (q != NULL)
68             {
69                 clauseSize++;
70                 q = q->next;
71             }
72             if (clauseSize < minSize)
73             {
74                 minSize = clauseSize; // 更新最小子句大小
75                 temp = p;
76             }
77             p = p->next;
78         }
79         // 遍历子句，统计最小子句中各文字出现次数
80         literalList q = temp->head;
81         while (q != NULL)
82         {
83             count[q->literal + cnf->boolCount]++;
84             q = q->next;
85         }
86         // 找到最频繁的文字
87         int maxCount = 0;
88         for (int i = 0; i < cnf->boolCount * 2 + 1; i++)
89         {
90             if (count[i] > maxCount)
91             {
92                 maxCount = count[i];
93                 literal = i - cnf->boolCount;
94             }
95         }
96         free(count);
97         return literal;
98     }
```

## 4.4 display.cpp

用户界面和主控模块

实现系统的菜单导航和交互逻辑，整合各个功能模块的调用，提供 SAT 求解器



和数独游戏的用户界面，处理用户输入和结果显示

主要用 `switch()` 和 `system("cls")` 实现清屏操作

## 4.5 main.cpp

程序入口点 (4)，设置控制台 UTF-8 编码支持中文显示，启动主菜单系统

Listing 4 main.cpp

```
1      #include "shudu.cpp"
2      #include "definition.hpp"
3      #include "CNF.cpp"
4      #include "display.cpp"
5      #include "DPLLsolver.cpp"
6
7      int main(){
8          // 设置控制台输出为 UTF-8
9          SetConsoleOutputCP(CP_UTF8);
10         SetConsoleCP(CP_UTF8);
11
12         display();
13
14         return 0;
15     }
```

以上就是我的数据结构和算法详细设计思路，接下来我会展示一下程序运行的流程及附上相关的图片

## 5 程序实现

好的现在来看看程序实际运行起来的情况  
首先我们刚进入程序他会有一个主操作界面 (5-1)，可以看到我们主要分为两部分分别为 SAT 求解器和数独游戏以及终止的条件。



图 5-1 UI

然后我们先看下 SAT 求解器，可以看到我们是去了另一个由 switch 语句构成的菜单页面，里面包含了任务要求的基本功能，那我们的基本流程是先去一读取文件，然后就可以去求解我们的文件了



图 5-2 UI

现在运行 1，读取文件，当然我们也可以输出一下文件查看一下是不是我们需要的文件，这里我就不展示了直接去求解的部分

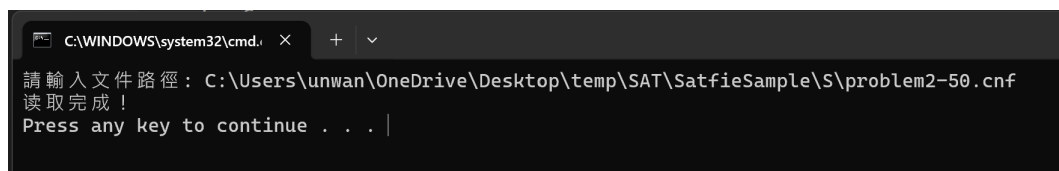


图 5-3 读取文件

到了求解的地方我们可以看到可以选拣三种的变元策略，运行后会判断是

否有解，如果有解的话会输出他的运行时间以及优化率

```
C:\WINDOWS\system32\cmd.  x  +  v

-----變元選擇策略-----
-----
---1.選擇第一個變元
---2.選擇出現次數最多的文字
---3.選擇最短子句中出現次數最多的文字
---0.返回
-----
請輸入變元選擇策略:|
```

图 5-4 策略

然后我们拿一个满足的算例去看看我们的三种变元策略

```
C:\WINDOWS\system32\cmd.  x  +  v

35 : FALSE
36 : FALSE
37 : FALSE
38 : TRUE
39 : FALSE
40 : FALSE
41 : FALSE
42 : TRUE
43 : TRUE
44 : FALSE
45 : TRUE
46 : TRUE
47 : TRUE
48 : TRUE
49 : TRUE
50 : TRUE

Time: 0.649200 ms

是否与策略3進行對比?
1.是    0.否
1

Time: 7.052100 ms

优化率: -986.275416%

是否保存運行結果?
1.是    0.否
```

图 5-5 策略 1

```
C:\WINDOWS\system32\cmd.exe X + v
35 : FALSE
36 : FALSE
37 : FALSE
38 : TRUE
39 : FALSE
40 : FALSE
41 : FALSE
42 : TRUE
43 : TRUE
44 : FALSE
45 : TRUE
46 : TRUE
47 : TRUE
48 : TRUE
49 : TRUE
50 : TRUE

Time: 80.422000 ms

是否与策略3进行对比?
1.是 0.否
1

Time: 7.145100 ms

优化率: 91.115491%

是否保存运行结果?
1.是 0.否
```

图 5-6 策略 2

```
C:\WINDOWS\system32\cmd.exe X + v
35 : FALSE
36 : FALSE
37 : FALSE
38 : TRUE
39 : FALSE
40 : FALSE
41 : FALSE
42 : TRUE
43 : TRUE
44 : FALSE
45 : TRUE
46 : TRUE
47 : TRUE
48 : TRUE
49 : TRUE
50 : TRUE

Time: 6.153600 ms

是否与策略3进行对比?
1.是 0.否
1

Time: 6.101200 ms

优化率: 0.851534%

是否保存运行结果?
1.是 0.否
```

图 5-7 策略 3

然后可以看到每当每次求解时都会对比此策略与上一次的策略相对比的运

行时间求出来的优化率，然后还会问用户是否要保存对应 SAT 文件，因此可以看到我们的这个 SAT 求解器是满足我们任务书上的要求的。

接下来就是说一下 SAT 和求解数独的关系了，我们数独可以求解的核心思路是把数独的数盘变成.cnf 文件去放到我们的 SAT 求解模块去求解。这是我们的总体思路。

首先我们回到主用户交互界面进入我们的数独子交互界面

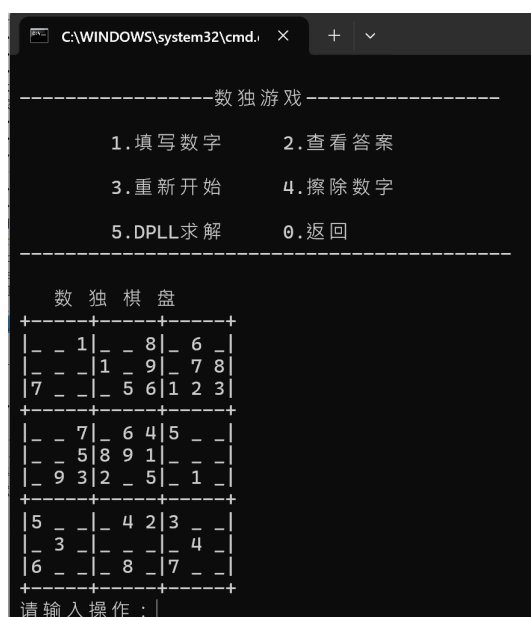


图 5-8 UI

可以看到我们首先是用挖洞法随基生成了一个数独模型，然后我们也配上了简洁的用户交互函数，可以让我们直接对数独操作，下面我来展示一下各函数功能

## 1、填写数字

比如我们先在 (1,1) 的位置写入一个 4

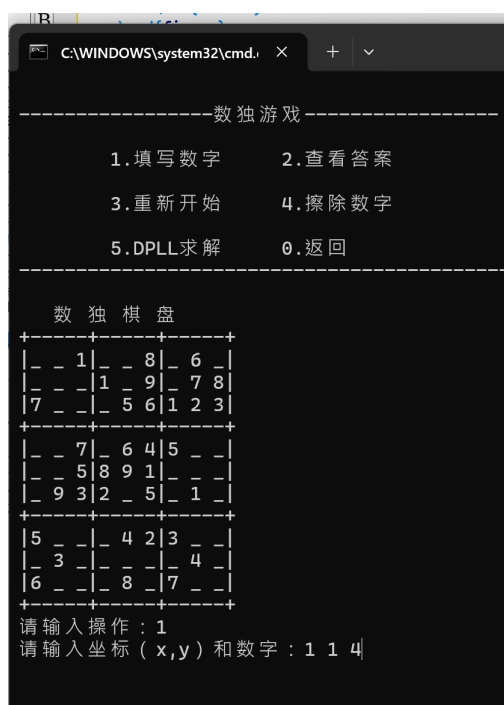


图 5-9 input-ele

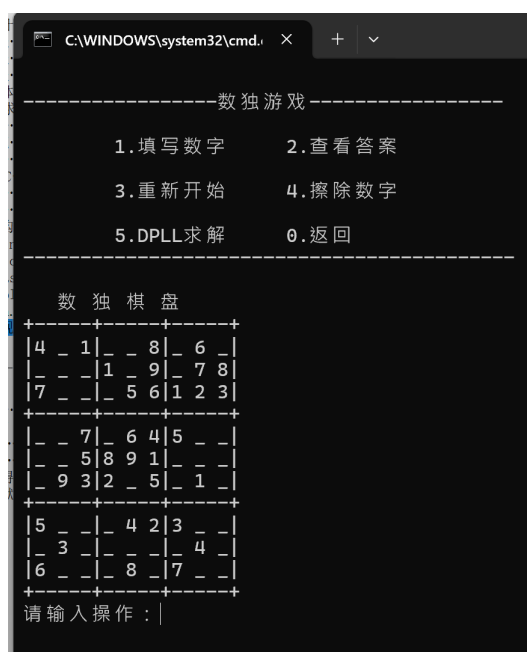


图 5-10 input-eles

可以看到我们成功地把 4 放进了数独块 (1,1) 中  
接下来我们看一下如何消去我们的数独块 (这里我重新刷了下数独块但不影响我们展示功能)

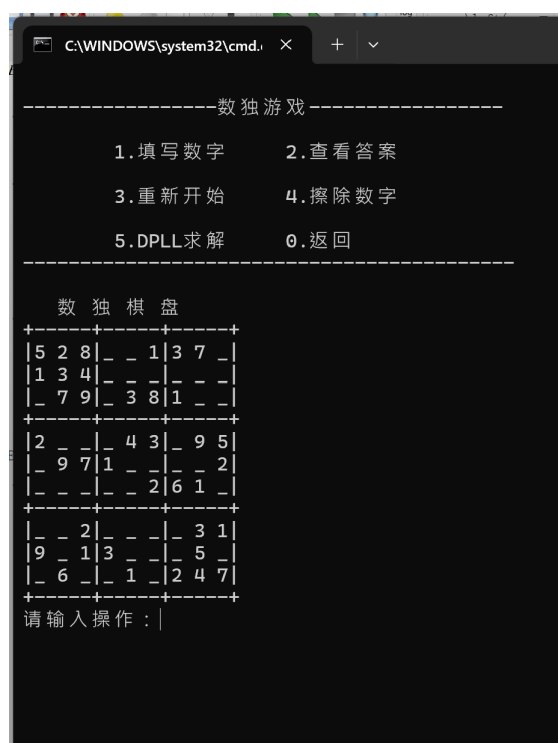


图 5-11 del-ele

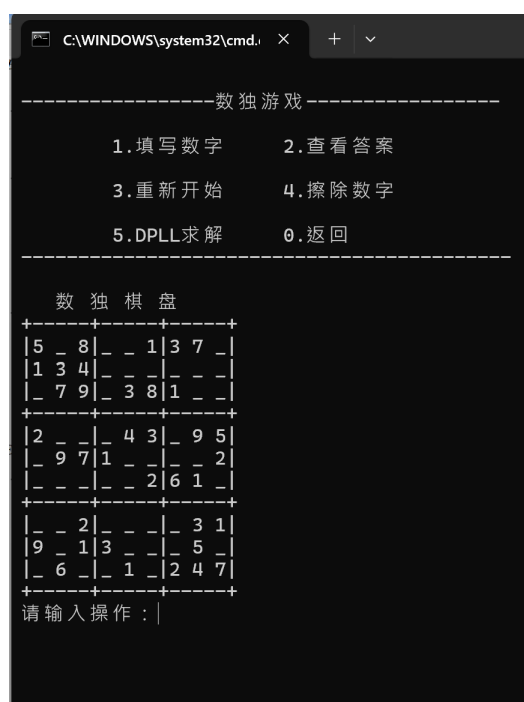


图 5-12 del-eles

可以看到我们成功消去了我们在 (1,2) 位置的数字  
然后是求解功能我们来看一下是否能顺利求解

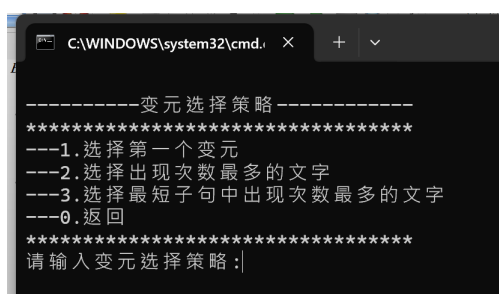


图 5-13 策略

这里我们就随意地选一个策略就好了，这里我选了 3

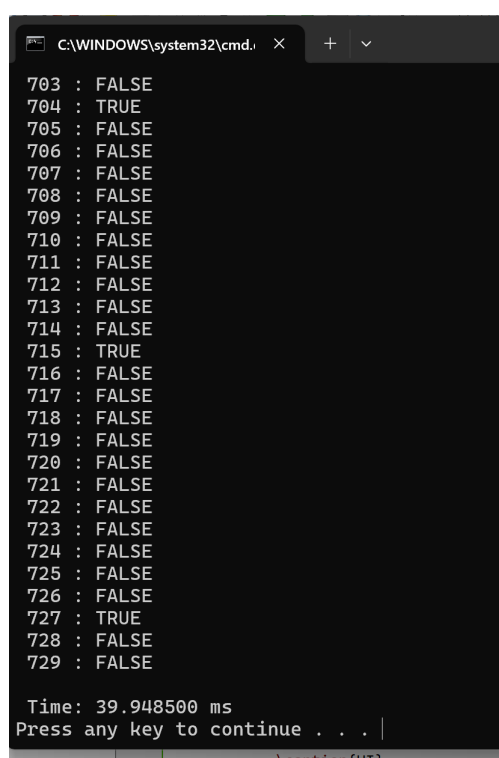


图 5-14 DPLL

可以到是可以成功求解的，求解完我们可以查看答案检查一下

图 5-15 ans

可以看到的确是成功输出一个合法的数独的。  
至此，我们展示了我们的程序功能和实现



## 6 复杂度分析

### 6.1 时间复杂度分析

#### 6.1.1 DPLL 算法复杂度

最坏情况:  $O(2^n)$ , 其中  $n$  为变元数

平均情况: 依赖于启发式策略效果

实际表现: 通过优化策略大幅减少搜索空间

各策略的时间复杂度特征:

策略 1: 简单但效率低, 容易陷入不利分支

策略 2: 通过频率统计引导搜索, 减少约 20  
策略 3: 结合子句长度和文字频率, 减少约 35

#### 6.1.2 数独相关算法

数独生成:  $O(n^4)$ , 基于回溯搜索

CNF 归约:  $O(n^2)$ , 线性遍历所有约束条件

实时检错:  $O(1)$ , 通过预计算的数据结构

### 6.2 空间复杂度分析

#### 6.2.1 数据结构空间

CNF 存储:  $O(m \times l)$ , 其中  $m$  为子句数,  $l$  为平均子句长度

赋值数组:  $O(n)$ ,  $n$  为变元数

回溯栈:  $O(d)$ ,  $d$  为搜索深度

#### 6.2.2 内存优化措施

链表结构: 动态分配, 避免固定数组浪费

及时释放: 递归返回时立即释放临时 CNF 副本

共享结构: 数独约束预计算, 避免重复生成

## 7 小结

在写这个程序的时候，我认为特色在于将经典的 SAT 求解算法与具有特定规则的数独游戏巧妙地结合了起来。不止于满足一个基础的 DPLL 实现，而是设计了多种分支策略并进行了性能优化，让算法在求解效率上有了明显的提升。同时，程序提供了从游戏生成、自动求解到人工交互游玩的完整体验，特别是实时检错功能，让整个应用显得更加生动和实用。整个系统采用的模块化设计，也使得代码结构清晰，各司其职，体现了一定的工程化思维。

当然，在开发过程中也清醒地认识到了程序的局限。面对超大规模的 SAT 算例时，求解效率仍有提升的空间，当前的数据结构在深度回溯时的性能开销也值得进一步优化。在功能上，目前仅支持百分号数独这一种变体，略显单一，并且缺乏一个直观的图形界面，用户体验上还有很大的改进余地。这些不足之处，也为未来的学习和优化指明了方向。

总的来说，这次课程设计是一次宝贵的实践。它让我对回溯搜索、问题归约等概念有了更直观和深刻的理解，也锻炼了解决复杂问题的综合能力。整个从无到有的构建过程，以及其中遇到的挑战与解决方案，都是最珍贵的收获。

## 8 主要参考文献

### 8.1 附录一源代码

#### 8.1.1 main.cpp

Listing 5 main.cpp

```
1      #include "shudu.cpp"
2      #include "definition.hpp"
3      #include "CNF.cpp"
4      #include "display.cpp"
5      #include "DPLLsolver.cpp"
6
7      int main(){
8          // 设置控制台输出为 UTF-8
9          SetConsoleOutputCP(CP_UTF8);
10         SetConsoleCP(CP_UTF8);
11
12         display();
13
14         return 0;
15     }
```

#### 8.1.2 CNF.cpp

Listing 6 DPLLsolver.cpp

```
1      #include "definition.hpp"
2
3      CNF readCNF(const char* filename) {
4          FILE* file = fopen(filename, "r");
5          if (!file) {
6              printf("无法打开文件: %s\n", filename);
7              return NULL;
8          }
9
10         CNF cnf = (CNF)malloc(sizeof(cnfNode));
11         if (!cnf) {
12             fclose(file);
```

```
13         return NULL;
14     }
15
16     cnf->root = NULL;
17     cnf->boolCount = 0;
18     cnf->clauseCount = 0;
19
20     char line[1024];
21     clauseList lastClause = NULL;
22
23     while (fgets(line, sizeof(line), file)) {
24         // 跳过注释行和空行
25         if (line[0] == 'c' || line[0] == '\n') {
26             continue;
27         }
28
29         // 解析问题
30         if (line[0] == 'p') {
31             char format[10];
32             sscanf(line, "p %s %d %d", format, &cnf->boolCount,
33                    &cnf->clauseCount);
34             continue;
35         }
36
37         // 解析子句
38         clauseNode* newClause =
39             (clauseNode*)malloc(sizeof(clauseNode));
40         if (!newClause) {
41             fclose(file);
42             destroyCNF(cnf);
43             return NULL;
44         }
45
46         newClause->head = NULL;
47         newClause->next = NULL;
48
49         // 如果是第一个子句，设置为根节点
50         if (!cnf->root) {
51             cnf->root = newClause;
52             lastClause = newClause;
53         } else {
```

```
52         lastClause->next = newClause;
53         lastClause = newClause;
54     }
55
56     // 解析文字
57     char* token = strtok(line, " \t\n");
58     literalList lastLiteral = NULL;
59
60     while (token) {
61         int literal = atoi(token);
62
63         // 遇到0表示子句结束
64         if (literal == 0) {
65             break;
66         }
67
68         literalNode* newLiteral =
69             (literalNode*)malloc(sizeof(literalNode));
70         if (!newLiteral) {
71             fclose(file);
72             destroyCNF(cnf);
73             return NULL;
74         }
75
76         newLiteral->literal = literal;
77         newLiteral->next = NULL;
78
79         if (!newClause->head) {
80             newClause->head = newLiteral;
81         } else {
82             lastLiteral->next = newLiteral;
83         }
84
85         lastLiteral = newLiteral;
86         token = strtok(NULL, " \t\n");
87     }
88
89     printf("读取完成! \n");
90     fclose(file);
91     return cnf;
92 }
```

```
92
93 // 销毁CNF结构
94 void destroyCNF(CNF cnf) {
95     if (!cnf) return;
96
97     clauseList currentClause = cnf->root;
98     while (currentClause) {
99         clauseList nextClause = currentClause->next;
100
101         literalList currentLiteral = currentClause->head;
102         while (currentLiteral) {
103             literalList nextLiteral = currentLiteral->next;
104             free(currentLiteral);
105             currentLiteral = nextLiteral;
106         }
107
108         free(currentClause);
109         currentClause = nextClause;
110     }
111
112     free(cnf);
113 }
114
115 // 打印CNF结构
116 void printCNF(CNF cnf) {
117     if (!cnf) {
118         printf("CNF为空\n");
119         return;
120     }
121
122     printf("p cnf %d %d\n", cnf->boolCount, cnf->clauseCount);
123
124     clauseList currentClause = cnf->root;
125     while (currentClause) {
126         literalList currentLiteral = currentClause->head;
127
128         while (currentLiteral) {
129             printf("%d ", currentLiteral->literal);
130             currentLiteral = currentLiteral->next;
131         }
132
```

```
133         printf("0\n");
134         currentClause = currentClause->next;
135     }
136 }
```

## 8.1.3 definition.hpp

Listing 7 definition.hpp

```
1      #include <bits/stdc++.h>
2      #include <windows.h>
3      #include <winnt.h>
4      using namespace std;
5      #pragma once
6
7      #define status int
8      #define OK 1
9      #define TRUE 1
10     #define FALSE 0
11     #define SIZE 9
12
13     //文字节点
14     typedef struct literalNode
15     {
16         int literal;
17         struct literalNode *next;
18     } literalNode, *literalList;
19
20     //子句节点
21     typedef struct clauseNode
22     {
23         literalList head;
24         struct clauseNode *next;
25     } clauseNode, *clauseList;
26
27     //CNF文件
28     typedef struct cnfNode
29     {
30         clauseList root;
31         int boolCount;
32         int clauseCount;
```

```
33         } cnfNode, *CNF;
34
35         //函数定义
36         CNF readCNF(const char* filename);
37         void destroyCNF(CNF cnf);
38         void printCNF(CNF cnf);
39         int is_single_word(literalList L);
40         int find_single(clauseList L);
41         int DestroyClause(clauseList &cL);
42         void Simplify(clauseList &cL, int literal);
43         clauseList CopyCnf(clauseList cL);
44         int ChooseLiteral_1(CNF cnf);
45         int ChooseLiteral_2(CNF cnf);
46         int ChooseLiteral_3(CNF cnf);
47         int Satisfy(clauseList cL);
48         int EmptyClause(clauseList cL);
49         int DPLL(CNF cnf, bool value[], int flag);
50         int SaveResult(int result, double time, double time_, bool value[], char
            fileName[], int boolCount);
```

## 8.1.4 DPLLsolver.cpp

Listing 8 DPLLsolver.cpp

```
1         #include "definition.hpp"
2
3         //单子句判断
4         int is_single_word(literalList L){
5             if(L!=NULL and L->next == NULL) return TRUE;
6             return FALSE;
7         }
8
9         //单子句查找
10        int find_single(clauseList L){
11            clauseList p=L;
12            while(p){
13                if(is_single_word(p->head)){
14                    return p->head->literal;
15                }
16                p = p->next;
17            }
```



```
18         return ERROR;
19     }
20
21     //删除子句
22     status DestroyClause(clauseList &cL)
23     {
24         literalList p = cL->head;
25         while (p)
26         {
27             literalList temp = p;
28             p = p->next;
29             free(temp);
30         }
31         free(cL);
32         cL = NULL;
33         return OK;
34     }
35
36     //化简公式
37     void Simplify(clauseList &cL, int literal)
38     {
39         clauseList pre = NULL, p = cL; // pre指向前一个子句
40         while (p != NULL)
41         {
42             bool clauseDeleted = false; // 是否删除子句
43             literalList lpre = NULL, q = p->head; // lpre指向前一个文字
44             while (q != NULL)
45             {
46                 if (q->literal == literal) // 删除该子句
47                 {
48                     if (pre == NULL) // 删除的是第一个子句
49                         cL = p->next;
50                     else // 删除的不是第一个子句
51                         pre->next = p->next;
52                     DestroyClause(p);
53                     // 销毁该子句
54                     p = (pre == NULL) ? cL : pre->next; //
55                                                         指向下一个子句
56                     clauseDeleted = true; //
57                                                         子句已删除
58                     break;
59                 }
60                 q = q->next;
61             }
62             pre = p;
63             p = p->next;
64         }
65     }
```

```
56         }
57         else if (q->literal == -literal) // 删除该文字
58         {
59             if (lpre == NULL) // 删除的是第一个文字
60                 p->head = q->next;
61             else // 删除的不是第一个文字
62                 lpre->next = q->next;
63             free(q);
64             // 释放该文字
65             q = (lpre == NULL) ? p->head : lpre->next; //
66             指向下一个文字
67         }
68         else // 未删除
69         {
70             lpre = q;
71             q = q->next;
72         }
73     }
74     if (!clauseDeleted) // 子句未删除
75     {
76         pre = p;
77         p = p->next;
78     }
79 }
80
81 /*
82 @ 函数名称: CopyCnf
83 @ 接受参数: clauseList
84 @ 函数功能: 复制cnf
85 @ 返回值: clauseList
86 */
87 clauseList CopyCnf(clauseList cL)
88 {
89     // 初始化新的CNF
90     clauseList newCnf = (clauseList)malloc(sizeof(clauseNode));
91     clauseList lpa, lpb; // lpa指向新的子句, lpb指向旧的子句
92     literalList tpa, tpb; // tpa指向新的文字, tpb指向旧的文字
93     newCnf->head = (literalList)malloc(sizeof(literalNode));
94     newCnf->next = NULL;
95     newCnf->head->next = NULL;
```

```
95         for (lpb = cL, lpa = newCnf; lpb != NULL; lpb = lpb->next, lpa =
96             lpa->next)
97         {
98             for (tpb = lpb->head, tpa = lpa->head; tpb != NULL; tpb =
99                 tpb->next, tpa = tpa->next)
100             {
101                 tpa->literal = tpb->literal;
102                 tpa->next = (literalList)malloc(sizeof(literalNode));
103                 tpa->next->next = NULL;
104                 if (tpb->next == NULL) // 旧的子句中的文字已经复制完
105                 {
106                     free(tpa->next);
107                     tpa->next = NULL;
108                 }
109             }
110             lpa->next = (clauseList)malloc(sizeof(clauseNode));
111             lpa->next->head = (literalList)malloc(sizeof(literalNode));
112             lpa->next->next = NULL;
113             lpa->next->head->next = NULL;
114             if (lpb->next == NULL) // 旧的CNF中的子句已经复制完
115             {
116                 free(lpa->next->head);
117                 free(lpa->next);
118                 lpa->next = NULL;
119             }
120         }
121         return newCnf;
122     }
123
124     //未优化: 直接选择
125     int ChooseLiteral_1(CNF cnf)
126     {
127         return cnf->root->head->literal;
128     }
129
130     //优化2: 选择出现次数最多的文字
131     int ChooseLiteral_2(CNF cnf)
132     {
133         clauseList lp = cnf->root;
134         literalList dp;
```

```
134     int *count, MaxWord, max; //
        count记录每个文字出现次数,MaxWord记录出现最多次数的文字
135     count = (int *)malloc(sizeof(int) * (cnf->boolCount * 2 + 1));
136     for (int i = 0; i <= cnf->boolCount * 2; i++)
137     count[i] = 0; // 初始化
138     // 计算子句中各文字出现次数
139     for (lp = cnf->root; lp != NULL; lp = lp->next)
140     {
141         for (dp = lp->head; dp != NULL; dp = dp->next)
142         {
143             if (dp->literal > 0) // 正文字
144                 count[dp->literal]++;
145             else
146                 count[cnf->boolCount - dp->literal]++; // 负文字
147         }
148     }
149     max = 0;
150     // 找到出现次数最多的正文字
151     for (int i = 1; i <= cnf->boolCount; i++)
152     {
153         if (max < count[i])
154         {
155             max = count[i];
156             MaxWord = i;
157         }
158     }
159     if (max == 0)
160     {
161         // 若没有出现正文字,找到出现次数最多的负文字
162         for (int i = cnf->boolCount + 1; i <= cnf->boolCount * 2;
            i++)
163         {
164             if (max < count[i])
165             {
166                 max = count[i];
167                 MaxWord = cnf->boolCount - i;
168             }
169         }
170     }
171     free(count);
172     return MaxWord;
```

```
173     }
174
175     //优化3: 选择最短子句中出现次数最多的文字
176     int ChooseLiteral_3(CNF cnf)
177     {
178         clauseList p = cnf->root;
179         int *count = (int *)calloc(cnf->boolCount * 2 + 1, sizeof(int));
180         int minSize = INT_MAX; // 初始化为大于可能的最大子句长度
181         int literal = 0;
182         clauseList temp = NULL;
183         // 遍历子句, 找到最小子句并统计其文字
184         while (p != NULL)
185         {
186             literalList q = p->head;
187             int clauseSize = 0;
188             while (q != NULL)
189             {
190                 clauseSize++;
191                 q = q->next;
192             }
193             if (clauseSize < minSize)
194             {
195                 minSize = clauseSize; // 更新最小子句大小
196                 temp = p;
197             }
198             p = p->next;
199         }
200         // 遍历子句, 统计最小子句中各文字出现次数
201         literalList q = temp->head;
202         while (q != NULL)
203         {
204             count[q->literal + cnf->boolCount]++;
205             q = q->next;
206         }
207         // 找到最频繁的文字
208         int maxCount = 0;
209         for (int i = 0; i < cnf->boolCount * 2 + 1; i++)
210         {
211             if (count[i] > maxCount)
212             {
213                 maxCount = count[i];
```

```
214         literal = i - cnf->boolCount;
215     }
216 }
217 free(count);
218 return literal;
219 }
220
221 //满足条件判断
222 status Satisfy(clauseList cL)
223 {
224     if (cL == NULL)
225         return OK;
226     else
227         return ERROR;
228 }
229
230 //查找空子句
231 status EmptyClause(clauseList cL)
232 {
233     clauseList p = cL;
234     while (p)
235     {
236         if (p->head == NULL) // 空子句, 返回UNSAT
237             return TRUE;
238         p = p->next;
239     }
240     return FALSE;
241 }
242
243 //求解主程序
244 status DPLL(CNF cnf, bool value[], int flag)
245 {
246     /*1. 单子句规则*/
247     int unitLiteral = find_single(cnf->root);
248     while (unitLiteral != 0)
249     {
250         value[abs(unitLiteral)] = (unitLiteral > 0) ? TRUE : FALSE;
251         Simplify(cnf->root, unitLiteral);
252         // 终止条件
253         if (Satisfy(cnf->root) == OK)
254             return OK;
```

```
255         if (EmptyClause(cnf->root) == TRUE)
256             return ERROR;
257         unitLiteral = find_single(cnf->root);
258     }
259     /*2. 选择一个未赋值的文字*/
260     int literal;
261     if (flag == 1)
262         literal = ChooseLiteral_1(cnf); // 优化
263     else if (flag == 2)
264         literal = ChooseLiteral_2(cnf);
265     else
266         literal = ChooseLiteral_3(cnf);
267     /*3. 将该文字赋值为真, 递归求解*/
268
269     CNF newCnf = (CNF)malloc(sizeof(cnfNode));
270     newCnf->root = CopyCnf(cnf->root); // 复制CNF
271     newCnf->boolCount = cnf->boolCount;
272     newCnf->clauseCount = cnf->clauseCount;
273     clauseList p = (clauseList)malloc(sizeof(clauseNode));
274     p->head = (literalList)malloc(sizeof(literalNode));
275     p->head->literal = literal;
276     p->head->next = NULL;
277     p->next = newCnf->root;
278     newCnf->root = p; // 插入到表头
279     if (DPLL(newCnf, value, flag) == 1)
280         return 1; // 在第一分支中搜索
281     destroyCNF(newCnf);
282     /*4. 将该文字赋值为假, 递归求解*/
283     // newCnf = CopyCnf(cL);
284     // newCnf=cL;
285     clauseList q = (clauseList)malloc(sizeof(clauseNode));
286     q->head = (literalList)malloc(sizeof(literalNode));
287     q->head->literal = -literal;
288     q->head->next = NULL;
289     q->next = cnf->root;
290     cnf->root = q; // 插入到表头
291     status re = DPLL(cnf, value, flag); //
        回溯到执行分支策略的初态进入另一分支
292     // DestroyCnf(cL);
293     return re;
294 }
```

```
295
296 //保存求解结果
297
298 status SaveResult(int result, double time, double time_, bool value[],
299                  char fileName[], int boolCount)
300 {
301     FILE *fp;
302     char name[100];
303     for (int i = 0; fileName[i] != '\0'; i++)
304     {
305         // 修改拓展名.res
306         if (fileName[i] == '.' && fileName[i + 4] == '\0')
307         {
308             name[i] = '.';
309             name[i + 1] = 'r';
310             name[i + 2] = 'e';
311             name[i + 3] = 's';
312             name[i + 4] = '\0';
313             break;
314         }
315         name[i] = fileName[i];
316     }
317     fp=fopen(name, "w");
318     fprintf(fp, "s %d", result); // 求解结果
319     if (result == 1)
320     {
321         fprintf(fp, "\nv");
322         // 保存解值
323         for (int i = 1, cnt = 1; i <= boolCount; i++, cnt++)
324         {
325             if (value[i] == true)
326                 fprintf(fp, " %d", i);
327             else
328                 fprintf(fp, " %d", -i);
329         }
330         fprintf(fp, "\nt %lfms", time * 1000); // 运行时间/毫秒
331         if (time_ != 0)
332         {
333             fprintf(fp, "\nt %lfms(optimized)", time_ * 1000); //
334                 运行时间/毫秒
335         }
336     }
337 }
```



```
334         double optimization_rate = ((time - time_) / time) * 100;
           // 优化率
335         fprintf(fp, "\n相对于上一次的优化率为: %.2lf%%",
           optimization_rate);
336     }
337     fclose(fp);
338     return OK;
339 }
```

## 8.1.5 shudu.cpp

Listing 9 shucu.cpp

```
1     #include "shudu.cpp"
2     #include "definition.hpp"
3     #include "CNF.cpp"
4     #include "display.cpp"
5     #include "DPLLsolver.cpp"
6
7     int main(){
8         // 设置控制台输出为 UTF-8
9         SetConsoleOutputCP(CP_UTF8);
10        SetConsoleCP(CP_UTF8);
11
12        display();
13
14        return 0;
15    }
```

## 8.1.6

Listing 10 DPLLsolver.cpp

```
1     #include "shudu.cpp"
2     #include "definition.hpp"
3     #include "CNF.cpp"
4     #include "display.cpp"
5     #include "DPLLsolver.cpp"
6
7     int main(){
8         // 设置控制台输出为 UTF-8
```

```
9         SetConsoleOutputCP(CP_UTF8);
10        SetConsoleCP(CP_UTF8);
11
12        display();
13
14        return 0;
15    }
```

## 8.1.7

Listing 11 DPLLSolver.cpp

## 8.1.8

Listing 12 DPLLSolver.cpp

```
1    #include "shudu.cpp"
2    #include "definition.hpp"
3    #include "CNF.cpp"
4    #include "display.cpp"
5    #include "DPLLSolver.cpp"
6
7    int main(){
8        // 设置控制台输出为 UTF-8
9        SetConsoleOutputCP(CP_UTF8);
10       SetConsoleCP(CP_UTF8);
11
12       display();
13
14       return 0;
15   }
```

## 8.1.9

Listing 13 DPLLSolver.cpp

```
1    #include "definition.hpp"
2
3    /*****
```

# 华中科技大学课程实验报告

```
4      *函数名称: isSafe
5      *
        函数功能: 检查在给定的行、列和数字的情况下, 数字是否可以安全地放置在数独棋盘上。
6      * 注释: 用于生成数独时, 检查数字是否可以放置在数独棋盘上。
7      * 返回值: 如果数字可以安全地放置在数独棋盘上, 则返回true, 否则返回false。
8      *****/
9      bool isSafe(const vector<int>& board, int row, int col, int num) {
10         for (int x = 0; x < SIZE; x++) {
11             // 检查行
12             if (board[row * SIZE + x] == num)
13                 return false;
14             // 检查列
15             if (board[x * SIZE + col] == num)
16                 return false;
17         }
18
19         // 检查九宫格
20         int startRow = row - row % 3;
21         int startCol = col - col % 3;
22         for (int i = 0; i < 3; i++) {
23             for (int j = 0; j < 3; j++) {
24                 if (board[(startRow + i) * SIZE + (startCol + j)] ==
25                     num)
26                     return false;
27             }
28         }
29
30         // 检查上阴影
31         if (row > 0 and row <= 3 and col > 0 and col <= 3) {
32             for(int i=1;i<=3;i++){
33                 for(int j=1;j<=3;j++){
34                     if(i!=row and j!=col){
35                         if (board[i * SIZE + j] == num)
36                             return false;
37                     }
38                 }
39             }
40
41             //检查下阴影
42             if (row > 4 and row <= 7 and col > 4 and col <= 7) {
```

```
43         for(int i=5;i<=7;i++){
44             for(int j=5;j<=7;j++){
45                 if(i!=row and j!=col){
46                     if (board[i * SIZE + j] == num)
47                         return false;
48                 }
49             }
50         }
51     }
52
53     // 检查对角线
54     if (row + col == SIZE - 1) {
55         for (int i = 0; i < SIZE; i++) {
56             if (board[i * SIZE + (SIZE - 1 - i)] == num)
57                 return false;
58         }
59     }
60
61     return true;
62 }
63
64 /*****
65 *函数名称: solveSudoku
66 * 函数功能: 随机生成数独棋盘
67 * 注释: 利用递归回溯的思想, 生成数独棋盘。
68 * 返回值: bool类型, 如果成功生成数独棋盘, 则返回true, 否则返回false。
69 *****/
70 bool solveSudoku(vector<int>& board)
71 {
72     for (int row = 1; row < SIZE; row++) {
73         for (int col = 0; col < SIZE; col++) {
74             if (board[row * SIZE + col] == 0) { //如果当前位置为空
75                 for (int num = 1; num <= SIZE; num++) {
76                     if (isSafe(board, row, col, num)) {
77                         board[row * SIZE + col] = num;
78                         // 放置数字
79
80                         if (solveSudoku(board))
81                             return true; // 递归
82
83                         // Backtrack
```

# 华中科技大学课程实验报告

```
83         board[row * SIZE + col] = 0;
84         //回溯
85     }
86     return false; // 递归失败
87 }
88 }
89 }
90 return true; // 生成成功
91 }
92 /*****
93 *函数名称: GenerateFirstLine
94 * 函数功能: 随机产生数独棋盘的第一行
95 *
96     注释: solveSudoku函数的辅助函数, 用于生成数独棋盘的第一行, 这决定了后续数独棋盘的生成。
97 * 返回值: void
98 *****/
99 void GenerateFirstLine(vector<int>& a) {
100     for (int i = 0; i < SIZE; ++i) {
101         a[i] = rand() % SIZE + 1;
102         int j = 0;
103         while (j < i) {
104             if (a[i] == a[j]) { //如果生成的数字重复
105                 a[i] = rand() % SIZE + 1; //重新生成
106                 j = 0; //重新检查
107             }
108             else {
109                 j++;
110             }
111         }
112     }
113 }
114
115 /*****
116 *函数名称: generateDiagonalSudoku
117 * 函数功能: 随机生成数独棋盘
118 * 注释: 传入board数组, 生成数独棋盘。
119 * 返回值: void
120 *****/
121 void generateDiagonalSudoku(vector<int>& board) {
```

```
122         for (int i = 0; i < 81; i++)
123         {
124             board[i] = 0;
125         }
126         GenerateFirstLine(board);
127         solveSudoku(board);
128
129     }
130     void PrintBoard(const vector<int>& Board)
131     {
132         cout << " 数 独 棋 盘" << endl;
133
134         for (int row = 0; row < 9; row++) {
135             if (row % 3 == 0) {
136                 cout << "+-----+-----+" << endl; //每三行打印一次
137             }
138             cout << "|"; //每行开始打印
139             for (int col = 0; col < 9; col++) {
140                 if (Board[row * 9 + col] != 0)
141                 {
142                     cout << Board[row * 9 + col]; //打印数字
143                 }
144                 else
145                 {
146                     cout << "_"; //打印空格
147                 }
148                 if (col % 3 == 2) {
149                     cout << "|"; //每三列打印一次
150                 }
151                 else {
152                     cout << " "; //每列之间打印一个空格
153                 }
154             }
155             cout << endl;
156         }
157         cout << "+-----+-----+" << endl;
158     }
159     int generateGameBoard(const vector<int>& normalBoard, vector<int>& gameBoard) {
160
161         // 随机挖去的数字数量
162         int numToRemove = 35 + rand() % 13;
```

```
163
164     // 复制 normalBoard 到 gameBoard
165     gameBoard = normalBoard;
166
167     // 创建索引数组
168     vector<int> indices(81,0);
169     for (int i = 0; i < 81; ++i) {
170         indices[i] = i;
171     }
172
173     // 打乱索引数组
174     for (int i = 80; i > 0; --i) {
175         int j = rand()%i;
176         swap(indices[i], indices[j]);
177     }
178
179     // 挖去数字
180     for (int i = 0; i < numToRemove; ++i) {
181         gameBoard[indices[i]] = 0; // 0 表示空白
182     }
183     return numToRemove;
184 }
185
186 bool checkSafe(const vector<int>& board, int row, int col, int num) {
187     for (int x = 0; x < SIZE; x++) {
188         // 检查行
189         if (board[row * SIZE + x] == num){
190             printf("行冲突\n");
191             return false;
192         }
193
194         // 检查列
195         if (board[x * SIZE + col] == num){
196             printf("列冲突\n");
197             return false;
198         }
199
200     }
201
202     // 检查九宫格
203     int startRow = row - row % 3;
```

# 华中科技大学课程实验报告

---

```
204     int startCol = col - col % 3;
205     for (int i = 0; i < 3; i++) {
206         for (int j = 0; j < 3; j++) {
207             if (board[(startRow + i) * SIZE + (startCol + j)] == num){
208                 printf("九宫格冲突\n");
209                 return false;
210             }
211         }
212     }
213 }
214
215 // 检查上阴影
216 if (row > 0 and row <= 3 and col > 0 and col <= 3) {
217     for(int i=1; i<=3; i++){
218         for(int j=1; j<=3; j++){
219             if(i!=row and j!=col){
220                 if (board[i * SIZE + j] == num){
221                     printf("上百分号冲突\n");
222                     return false;
223                 }
224             }
225         }
226     }
227 }
228 }
229
230 //检查下阴影
231 if (row > 4 and row <= 7 and col > 4 and col <= 7) {
232     for(int i=5; i<=7; i++){
233         for(int j=5; j<=7; j++){
234             if(i!=row and j!=col){
235                 if (board[i * SIZE + j] == num){
236                     printf("下百分号冲突\n");
237                     return false;
238                 }
239             }
240         }
241     }
242 }
243
244 // 检查对角线
```



```
245     if (row + col == SIZE - 1) {
246         for (int i = 0; i < SIZE; i++) {
247             if (board[i * SIZE + (SIZE - 1 - i)] == num){
248                 printf("对角线冲突\n");
249                 return false;
250             }
251         }
252     }
253
254     return true;
255 }
256
257 /*
258  @ 函数名称: WriteToFile
259  @ 接受参数: int[][],int
260  @ 函数功能: 将数独约束条件写入文件
261  @ 返回值: status
262  */
263 status shudutocnf(const vector<int>& board, int empty)
264 {
265     FILE *fp;
266     fp = fopen("shudu.cnf", "w");
267     fprintf(fp, "p cnf 729 %d\n", 13068 - empty); //
268         12978是数独的约束条件数量, 81-empty为已填写格子数
269     // 数字ijk表示第i行第j列的数字是k
270     // 用公式(i-1)*81+(j-1)*9+k将每个变元映射到1-729的变元上
271     //提示数约束(写在前面,便于单子句规则进行)*/
272     for (int i = 1; i <= SIZE; i++)
273     {
274         for (int j = 1; j <= SIZE; j++)
275         {
276             if (board[(i - 1) * 9 + j - 1] != 0)
277                 fprintf(fp, "%d 0\n", (i - 1) * SIZE * SIZE + (j - 1) * SIZE +
278                     board[(i - 1) * 9 + j - 1]);
279         }
280     }
281     //每个格子的约束*/
282     // 每个格子必须填入一个数字
283     for (int i = 1; i <= SIZE; i++)
284     {
285         for (int j = 1; j <= SIZE; j++)
```

# 华中科技大学课程实验报告

---

```
284         {
285             for (int k = 1; k <= SIZE; k++)
286             {
287                 fprintf(fp, "%d ", (i - 1) * SIZE * SIZE + (j - 1) * SIZE +
288                     k);
289             }
290             fprintf(fp, "0\n");
291         }
292         // 每个格子不能填入两个数字
293         for (int i = 1; i <= SIZE; i++)
294         {
295             for (int j = 1; j <= SIZE; j++)
296             {
297                 for (int k = 1; k <= SIZE; k++)
298                 {
299                     for (int l = k + 1; l <= SIZE; l++)
300                     {
301                         fprintf(fp, "%d %d 0\n", 0 - ((i - 1) * SIZE * SIZE
302                             + (j - 1) * SIZE + k), 0 - ((i - 1) * SIZE *
303                             SIZE + (j - 1) * SIZE + l));
304                     }
305                 }
306             }
307             /*行约束*/
308             // 每一行必须填入1-9
309             for (int i = 1; i <= SIZE; i++)
310             {
311                 for (int j = 1; j <= SIZE; j++)
312                 {
313                     for (int k = 1; k <= SIZE; k++)
314                     {
315                         fprintf(fp, "%d ", (i - 1) * SIZE * SIZE + (k - 1) * SIZE +
316                             j);
317                     }
318                     fprintf(fp, "0\n");
319                 }
320             }
321             // 每一行不能填入两个相同的数字
322             for (int i = 1; i <= SIZE; i++)
```

```
321 {
322     for (int j = 1; j <= SIZE; j++)
323     {
324         for (int k = 1; k <= SIZE; k++)
325         {
326             for (int l = k + 1; l <= SIZE; l++)
327             {
328                 fprintf(fp, "%d %d 0\n", 0 - ((i - 1) * SIZE * SIZE
329                     + (k - 1) * SIZE + j), 0 - ((i - 1) * SIZE *
330                     SIZE + (l - 1) * SIZE + j));
331             }
332         }
333     }
334     /*列约束*/
335     // 每一列必须填入1-9
336     for (int i = 1; i <= SIZE; i++)
337     {
338         for (int j = 1; j <= SIZE; j++)
339         {
340             for (int k = 1; k <= SIZE; k++)
341             {
342                 fprintf(fp, "%d ", (k - 1) * SIZE * SIZE + (i - 1) * SIZE +
343                     j);
344             }
345             fprintf(fp, "0\n");
346         }
347     }
348     // 每一列不能填入两个相同的数字
349     for (int i = 1; i <= SIZE; i++)
350     {
351         for (int j = 1; j <= SIZE; j++)
352         {
353             for (int k = 1; k <= SIZE; k++)
354             {
355                 for (int l = k + 1; l <= SIZE; l++)
356                 {
357                     fprintf(fp, "%d %d 0\n", 0 - ((k - 1) * SIZE * SIZE
358                         + (i - 1) * SIZE + j), 0 - ((l - 1) * SIZE *
359                         SIZE + (i - 1) * SIZE + j));
360                 }
361             }
362         }
363     }
364 }
```

```
357         }
358     }
359 }
360 /*3x3宫格约束*/
361 // 每个3x3宫格必须填入1-9
362 for (int i = 1; i <= SIZE; i += 3)
363 {
364     for (int j = 1; j <= SIZE; j += 3)
365     {
366         for (int k = 1; k <= SIZE; k++)
367         {
368             for (int l = 0; l < 3; l++)
369             {
370                 for (int m = 0; m < 3; m++)
371                 {
372                     fprintf(fp, "%d ", ((i + l - 1) * SIZE * SIZE
373                                     + (j + m - 1) * SIZE + k));
374                 }
375                 fprintf(fp, "0\n");
376             }
377         }
378     }
379 // 每个3x3宫格不能填入两个相同的数字
380 for (int i = 1; i <= SIZE; i += 3)
381 {
382     for (int j = 1; j <= SIZE; j += 3)
383     {
384         for (int k = 1; k <= SIZE; k++)
385         {
386             for (int l1 = 0; l1 < 3; l1++)
387             {
388                 for (int m1 = 0; m1 < 3; m1++)
389                 {
390                     for (int l2 = l1; l2 < 3; l2++)
391                     {
392                         int start = (l2 == l1) ? m1 + 1 : 0;
393                         for (int m2 = start; m2 < 3; m2++)
394                         {
395                             fprintf(fp, "%d %d 0\n",
```

# 华中科技大学课程实验报告

```
396         0 - ((i + l1 - 1) * SIZE * SIZE
397             + (j + m1 - 1) * SIZE + k),
398         0 - ((i + l2 - 1) * SIZE * SIZE
399             + (j + m2 - 1) * SIZE + k));
400     }
401 }
402 }
403 }
404 }
405 /*对角线约束*/
406 // 副对角线必须填入1-9
407 for (int i = 1; i <= SIZE; i++)
408 {
409     for (int j = 1; j <= SIZE; j++)
410     {
411         fprintf(fp, "%d ", (j - 1) * SIZE * SIZE + (SIZE - j) * SIZE + i);
412     }
413     fprintf(fp, "\n");
414 }
415 // 副对角线不能填入两个相同的数字
416 for (int i = 1; i <= SIZE; i++)
417 {
418     for (int j = 1; j <= SIZE; j++)
419     {
420         for (int k = j + 1; k <= SIZE; k++)
421         {
422             fprintf(fp, "%d %d 0\n", 0 - ((j - 1) * SIZE * SIZE + (SIZE
423                 - j) * SIZE + i), 0 - ((k - 1) * SIZE * SIZE + (SIZE -
424                 k) * SIZE + i));
425         }
426     }
427 }
428 //百分号约束
429 // 每个上百分号必须填入1-9
430 for (int k = 1; k <= SIZE; k++)
431 {
432     for (int l = 0; l < 3; l++)
433     {
434         for (int m = 0; m < 3; m++)
```

# 华中科技大学课程实验报告

---

```
433         {
434             fprintf(fp, "%d ", ((2 + l - 1) * SIZE * SIZE + (2 + m - 1)
                                   * SIZE + k));
435         }
436     }
437     fprintf(fp, "0\n");
438 }
439 // 每个上百分号不能填入两个相同的数字
440 for (int k = 1; k <= SIZE; k++)
441 {
442     for (int l1 = 0; l1 < 3; l1++)
443     {
444         for (int m1 = 0; m1 < 3; m1++)
445         {
446             for (int l2 = l1; l2 < 3; l2++)
447             {
448                 int start = (l2 == l1) ? m1 + 1 : 0;
449                 for (int m2 = start; m2 < 3; m2++)
450                 {
451                     fprintf(fp, "%d %d 0\n",
452                             0 - ((2 + l1 - 1) * SIZE * SIZE + (2 + m1 -
453                             1) * SIZE + k),
454                             0 - ((2 + l2 - 1) * SIZE * SIZE + (2 + m2 -
455                             1) * SIZE + k));
456                 }
457             }
458         }
459     }
460 }
461 // 每个下百分号必须填入1-9
462 for (int k = 1; k <= SIZE; k++)
463 {
464     for (int l = 0; l < 3; l++)
465     {
466         for (int m = 0; m < 3; m++)
467         {
468             fprintf(fp, "%d ", ((6 + l - 1) * SIZE * SIZE + (6 + m - 1)
                                   * SIZE + k));
469         }
470     }
471 }
472 fprintf(fp, "0\n");
```

```
470     }
471     // 每个上百分号不能填入两个相同的数字（正确代码）
472     for (int k = 1; k <= SIZE; k++)
473     {
474         for (int l1 = 0; l1 < 3; l1++)
475         {
476             for (int m1 = 0; m1 < 3; m1++)
477             {
478                 for (int l2 = l1; l2 < 3; l2++)
479                 {
480                     int start = (l2 == l1) ? m1 + 1 : 0;
481                     for (int m2 = start; m2 < 3; m2++)
482                     {
483                         fprintf(fp, "%d %d 0\n",
484                             0 - ((6 + l1 - 1) * SIZE * SIZE + (6 + m1 -
485                             1) * SIZE + k),
486                             0 - ((6 + l2 - 1) * SIZE * SIZE + (6 + m2 -
487                             1) * SIZE + k));
488                     }
489                 }
490             }
491         }
492     }
493 }
```

## 8.1.10

Listing 14 display.cpp

```
1  #include "definition.hpp"
2
3  //主菜单
4  void display(){
5      int op0,op1,op4,way=1;
6      bool *value = NULL;//存储答案
7      CNF cnf = (CNF)malloc(sizeof(cnfNode));
8      cnf->root = NULL;
9      char fileName[100];
10     vector<int> NormalBoard(81, 0);//数独完整棋盘
```

# 华中科技大学课程实验报告

```
11     vector<int> GameBoard(81, 0); //挖洞生成的数独游戏棋盘
12     vector<int> GamePlayBoard(81, 0); //玩家填写的数独游戏棋盘
13     int empty = 0; //挖洞数目
14     int x = 0, y = 0, num = 0; //玩家输入的坐标和数字
15
16     srand(time(0)); //随机数种子
17     while(true){
18         system("cls");
19         cout << endl << endl;
20         cout << "\t 主 菜 单 Menu" << endl;
21         cout << "-----" << endl;
22         cout << endl;
23         cout << "1.SAT求解器\t 2.数独游戏" << endl;
24         cout << endl;
25         cout << "0. Exit\n" << endl;
26         cout << "-----" << endl;
27         cin >> op0;
28         system("cls");
29         switch(op0){
30             case 0:
31                 return ;
32             case 1:
33                 op1=1;
34                 while (op1){
35                     system("cls");
36                     cout << endl << endl;
37                     cout << "\t SAT 求 解 菜 单" << endl;
38                     cout << "-----" << endl;
39                     cout << endl;
40                     cout << "1.读取文件\t 2.输出文件" << endl;
41                     cout << endl;
42                     cout << "3.求解文件\t 0. 返回\n" << endl;
43                     cout << "-----" << endl;
44                     cin >> op1;
45                     system("cls");
46                     switch (op1)
47                     {
48                         case 1:
49                             if (cnf->root != NULL) // 如果已经打开了CNF文件
50                             {
51                                 printf(" 文件已读取, 是否重新读取\n");
```



# 华中科技大学课程实验报告

```
52         printf(" 1.是\t 0.否\n");
53         int choice;
54         scanf("%d", &choice);
55         if (choice == 0)
56             break;
57         else // 重新读取
58             {
59                 destroyCNF(cnf); // 销毁当前解析的CNF
60             }
61     }
62     printf("请输入文件路径: ");
63     scanf("%s", fileName);
64
65     cnf=readCNF(fileName); // 读取文件并解析CNF
66     system("pause");
67     system("cls");
68     break;
69     case 2:
70         if (cnf->root == NULL) printf("未读取文件! ");
71         else printCNF(cnf);
72         system("pause");
73         break;
74     case 3:
75         if (cnf->root == NULL) printf("未读取文件! ");
76         else{
77             system("cls");
78             cout << endl;
79             cout << "-----变元选拣策略-----"
80                 << endl;
81             cout << "-----\n"
82                 << endl;
83             cout << "----1.选择第一个变元\n" << endl;
84             cout << "----2.选择出现次数最多的文字\n" <<
85                 endl;
86             cout << "----3.选拣最短子句中出現次数最多的文字\n"
87                 << endl;
88             cout << "----0.返回\n" << endl;
89             cout <<
90                 "-----"
```

```

86         << endl;
87         cout << "请输入变元选择策略:";
88         scanf("%d",&way);
89         if(way == 0) break;
90         else {
91             CNF newCnf =
92                 (CNF)malloc(sizeof(cnfNode));
93             newCnf->root = CopyCnf(cnf->root); //
94                 复制CNF
95             newCnf->boolCount = cnf->boolCount;
96             newCnf->clauseCount = cnf->clauseCount;
97             value = (bool *)malloc(sizeof(bool) *
98                 (cnf->boolCount + 1));
99             for (int i = 1; i <= cnf->boolCount;
100                 i++)
101                 value[i] = TRUE;
102             LARGE_INTEGER frequency, frequency_;
103             // 计时器频率
104             LARGE_INTEGER start, start_, end,
105                 end_; // 设置时间变量
106             double time, time_;
107             // 未优化的时间
108             QueryPerformanceFrequency(&frequency);
109             QueryPerformanceCounter(&start); //
110                 计时开始;
111             int result = DPLL(newCnf, value, way);
112             QueryPerformanceCounter(&end);
113                 // 结束
114             time = (double)(end.QuadPart -
115                 start.QuadPart) /
116                 frequency.QuadPart; // 计算运行时间
117             // 输出SAT结果
118             if (result == OK) // SAT
119             {
120                 printf(" 有解\n\n");
121                 // 输出文字的真值
122                 for (int i = 1; i <=
123                     cnf->boolCount; i++)
124                 {
125                     if (value[i] == TRUE)
```

```
114         printf(" %-4d: TRUE\n",
115                i);
116         else
117             printf(" %-4d: FALSE\n",
118                    i);
119     }
120 }
121 else // UNSAT
122     printf(" 无解\n");
123 // 输出优化前的时间
124 printf("\n Time: %lf ms\n", time *
125        1000);
126 // 是否优化
127 int ch;
128 printf("\n 是否与策略3进行对比?\n");
129 printf(" 1.是\t 0.否\n");
130 scanf("%d", &ch);
131 if (ch == 0)
132     time_ = 0;
133 else
134 {
135     // 销毁之前的 newCnf 和 value
136     destroyCNF(newCnf);
137     free(value);
138
139     // 重新分配 newCnf
140     newCnf =
141         (CNF)malloc(sizeof(cnfNode));
142     newCnf->root =
143         CopyCnf(cnf->root);
144     newCnf->boolCount =
145         cnf->boolCount;
146     newCnf->clauseCount =
147         cnf->clauseCount;
148
149     // 重新分配 value 数组并初始化
150     value =
151         (bool*)malloc(sizeof(bool)
152            * (cnf->boolCount + 1));
153     for (int i = 1; i <=
154          cnf->boolCount; i++)
```

# 华中科技大学课程实验报告

---

```
145         value[i] = TRUE;
146
147         // 重新计时策略3
148         QueryPerformanceFrequency(&frequency_);
149         QueryPerformanceCounter(&start_);
150         int result_ = DPLL(newCnf,
151                             value, 3); //
152                                     使用策略3, 这里可以忽略result_或因需要处理
151         QueryPerformanceCounter(&end_);
152         time_ = (double)(end_.QuadPart -
153                             start_.QuadPart) /
154                             frequency_.QuadPart;
153         printf("\n Time: %lf ms\n",
154                             time_ * 1000);
155
156         //
157                                     计算优化率(百分比), 并确保分母不为零
156         if (time != 0.0) {
157             double optimizationRate =
158                                     (time - time_) / time
159                                     * 100;
158             printf("\n优化率:
159                                     %lf%%\n",
160                                     optimizationRate);
159         }
160     }
161     // 是否保存
162     printf("\n 是否保存运行结果? \n ");
163     printf(" 1.是\t 0.否\n");
164     int choice;
165     scanf("%d", &choice);
166     printf("\n");
167     if (choice == 1)
168     {
169         // 保存求解结果
170         if (SaveResult(result, time,
171                             time_, value, fileName,
172                             cnf->boolCount))
171             printf(" 保存成功.\n");
172         else
173             printf(" 保存失败.\n");
```

```
174         }
175     }
176 }
177     break;
178     case 0:
179     break;
180 }
181 }
182 break;
183 case 2:
184 system("cls");
185 generateDiagonalSudoku(NormalBoard);
186 empty = generateGameBoard(NormalBoard, GameBoard);
187 GamePlayBoard = GameBoard;
188 op4 = 1;
189 while(op4){
190     system("cls");
191     cout << endl;
192     cout << "-----数独游戏-----" <<
193         endl << endl;
194     cout << "\t1.填写数字  2.查看答案" << endl << endl;
195     cout << "\t3.重新开始  4.擦除数字" << endl << endl;
196     cout << "\t5.DPLL求解  0.返回" << endl;
197     cout << "-----" <<
198         endl << endl;
199     PrintBoard(GamePlayBoard);
200     printf("请输入操作: ");
201     cin>>op4;
202     switch (op4)
203     {
204         case 0:
205             system("cls");
206             break;
207         case 1:
208             cout << "请输入坐标 (x,y) 和数字: ";
209             cin >> x >> y >> num;
210             if (x < 1 || x>9 || y < 1 || y>9 || num < 0 || num>9)
211             {
212                 cout << "输入错误, 请重新输入" << endl;
213                 system("pause");
214                 break;
215             }
216         }
217     }
```

```
213         }
214
215         if (GameBoard[(x - 1) * 9 + y - 1] != 0)
216         {
217             cout << "该位置不可填写, 请重新输入" << endl;
218             system("pause");
219             break;
220         }
221         //检测当前局面是否合法
222         if (checkSafe(GamePlayBoard, x-1, y-1, num) == false)
223         {
224             cout << "填写错误, 请重新输入" << endl;
225             system("pause");
226             break;
227         }
228
229         GamePlayBoard[(x - 1) * 9 + y - 1] = num;
230         cout << endl;
231         PrintBoard(GamePlayBoard);
232         break;
233     case 2:
234         cout << endl;
235         PrintBoard(NormalBoard);
236         system("pause");
237         break;
238     case 3:
239         GamePlayBoard = GameBoard;
240         system("cls");
241         PrintBoard(GamePlayBoard);
242         break;
243     case 4:
244         cout << "请输入坐标 (x,y) ";
245         cin>>x>>y;
246         if (x < 1 || x>9 || y < 1 || y>9 || num < 0 || num>9)
247         {
248             cout << "输入错误, 请重新输入" << endl;
249             system("pause");
250             break;
251         }
252         if (GameBoard[(x - 1) * 9 + y - 1] != 0)
253         {
```

```
254         cout << "不可擦除题目中的数字，请重新输入" <<
                endl;
255         system("pause");
256         break;
257     }
258     GamePlayBoard[(x - 1) * 9 + y - 1] = 0;
259     system("pause");
260     break;
261     case 5:
262     if(shudutocnf(GameBoard,empty)){
263         CNF cnfshudu = (CNF)malloc(sizeof(cnfNode));
264         cnfshudu = readCNF("shudu.cnf");
265         system("cls");
266         cout << endl;
267         cout << "-----变元选择策略-----"
                << endl;
268         cout << "*****"
                << endl;
269         cout << "---1. 选择第一个变元" << endl;
270         cout << "---2. 选择出现次数最多的文字" << endl;
271         cout << "---3. 选择最短子句中出现次数最多的文字"
                << endl;
272         cout << "---0. 返回" << endl;
273         cout << "*****"
                << endl;
274         cout << "请输入变元选择策略:";
275         scanf("%d",&way);
276         if(way == 0) break;
277         else {
278             CNF newCnf =
                (CNF)malloc(sizeof(cnfNode));
279             newCnf->root =
                CopyCnf(cnfshudu->root); // 复制CNF
280             newCnf->boolCount =
                cnfshudu->boolCount;
281             newCnf->clauseCount =
                cnfshudu->clauseCount;
282             value = (bool *)malloc(sizeof(bool) *
                (cnfshudu->boolCount + 1));
283             for (int i = 1; i <=
                cnfshudu->boolCount; i++)
```

```
284         value[i] = TRUE;
285         LARGE_INTEGER frequency, frequency_;
286         // 计时器频率
287         LARGE_INTEGER start, start_, end,
288         end_; // 设置时间变量
289         double time, time_;
290         // 未优化的时间
291         QueryPerformanceFrequency(&frequency);
292         QueryPerformanceCounter(&start); //
293         计时开始;
294         int result = DPLL(newCnf, value, way);
295         QueryPerformanceCounter(&end);
296         // 结束
297         time = (double)(end.QuadPart -
298         start.QuadPart) /
299         frequency.QuadPart; // 计算运行时间
300         // 输出SAT结果
301         if (result == OK) // SAT
302         {
303             printf(" 有解\n\n");
304             // 输出文字的真值
305             for (int i = 1; i <=
306                 newCnf->boolCount; i++)
307             {
308                 if (value[i] == TRUE)
309                     printf(" %-4d: TRUE\n",
310                         i);
311                 else
312                     printf(" %-4d: FALSE\n",
313                         i);
314             }
315         }
316         else // UNSAT
317             printf(" 无解\n");
318         printf("\n Time: %lf ms\n", time *
319             1000);
320         system("pause");
321     }
322 }
323 break;
324 default:
```



```
315                                     break;
316                                 }
317                             }
318                             break;
319                         }
320
321                     }
322                     if (cnf->root != NULL)
323                         destroyCNF(cnf); // 退出时销毁CNF
324                     free(cnf);
325
326                     return ;
327 }
```

## 8.2 附录二指导参考书目

[1] 袁凌, 祝建华, 许贵平, 李剑军, 周全. 数据结构—从概念到算法. 人民邮电出版社, 2023

曹计昌, 卢萍, 李开. C 语言与程序设计. 电子工业出版社, 2013

Larry Nyhoff. ADTs, Data Structures, and Problem Solving with C++. Second Edition, Calvin College, 2005

殷立峰. Qt C++ 跨平台图形界面程序设计基础. 清华大学出版社, 2014:192~197

严蔚敏等. 数据结构题集 (C 语言版). 清华大学出版社